Binary Tree Cameras

Company: Amazon
Round: Offline
Year: 2025
Difficulty: Medium-Hard to Hard
Topic: Binary Tree, DFS, Greedy, Tree DP
Source: https://www.desiqna.in/19233/amazon-sde-2-recent-interview-experiences-2026-set-66

Problem Description

Given a binary tree, we need to place cameras on some nodes.
Each camera can monitor:

- The node where it is placed
- Its parent
- Its immediate children

The goal is to find the **minimum number of cameras** required to monitor the complete tree.

Input Format

- The first line contains an integer N, representing the number of nodes in the binary tree.
- The next N lines describe each node.
- Each line contains three values:
  node left_child right_child

Output Format

We should **not place cameras on leaf nodes**.

A leaf can only monitor:

- itself
- its parent

Instead, it is better to place the camera on the **parent of the leaf**, because that camera can monitor more nodes.

Sample Input

root = [0, 0, null, 0, 0]

Sample Output

1

Explanation

Place **one camera on the second node** (the parent of the two leaf nodes).

That camera covers:

- The parent node
- Both leaf nodes
- Its own position

Therefore, the **minimum number of cameras required is 1**.

Approach to Solve This Problem:

1. Use Postorder DFS

First, check the **left child**, then the **right child**, and finally check the **current node**.
This helps us decide the best place to put a camera from the bottom of the tree upward.

2. Give Every Node a State

Each node can be in one of three conditions:

- 0 → The node **needs a camera**.
- 1 → The node **has a camera**.
- 2 → The node is **already covered** by a nearby camera.

3. Check the Children

After checking the children:

- If a child **needs a camera**, put a camera on the current node.
- If a child **has a camera**, the current node is already covered.
- If neither happens, the current node may need its parent to cover it.

4. Don't Put Cameras on Leaf Nodes

A leaf node is at the end of the tree. A camera there can cover only a few nodes.
It is better to put the camera on its **parent**, because the parent can cover the leaf and other nearby nodes.

5. Count the Cameras

Whenever we place a camera on a node, increase the camera count by 1.
At the end, this count gives the **minimum number of cameras needed**.

Java Solution:

```java
class Solution {
    int cameras = 0;
    int dfs(TreeNode node) {
        if (node == null) {
            return 2;
        }
        int left = dfs(node.left);
        int right = dfs(node.right);
        if (left == 0 || right == 0) {
            cameras++;
            return 1;
        }
        if (left == 1 || right == 1) {
            return 2;
        }
        return 0;
    }
    public int minCameraCover(TreeNode root) {
        if (dfs(root) == 0) {
            cameras++;
        }
        return cameras;
    }
}
```

Solution:

The main idea is to avoid placing cameras directly on leaf nodes because a camera on a leaf can cover only the leaf, its parent, and itself. Instead, it is better to place the camera on the parent of the leaf. This allows one camera to cover the parent, the leaf, its other child, and the parent's parent. We process the binary tree from bottom to top using postorder traversal. At each node, we check whether its children need a camera or are already covered. If a child needs a camera, we place a camera on the current node. By making these decisions from the bottom of the tree to the top, we avoid unnecessary cameras and find the minimum number of cameras needed to cover the entire tree.

Complexity Analysis:

Time Complexity: O(n)

Every node in the binary tree is visited once.

Space Complexity: O(h)

Due to the DFS recursion stack, where h is the height of the tree.